Academic Certificate Verification Using Blockchain

Improvements and Cryptographic Implementation

Problem Statement Introduction

Forged certificates undermine trust in qualifications. Manual verification is slow and depends on the issuing institution, while central databases can be breached or become unavailable.

The base paper stores certificate records in an Ethereum smart contract and certificate files on IPFS.

This project improves that design so anyone can check who issued a certificate and whether it has been altered or revoked, while students reveal only the details they choose.

Base paper

M. A. Patel, S. Sobhanayak and A. Sahoo, "Academic Certificate Verification using Blockchain Technology," IEEE InC4, 2025. DOI: 10.1109/InC465408.2025.11256194.

Cryptographic Implementations Used

Group	Implementation	Purpose
Hashing	SHA-256 for PDFs	Detects any change to the certificate file
Hashing	SHA-256 for tokens	Keeps sessions, claim codes and share links as hashes only
Hashing	scrypt	Makes password guessing expensive
Encryption	AES-256-GCM for PDFs	Encrypts files before IPFS upload and detects tampering
Encryption	AES-256-GCM for stored data	Protects credentials in a copied database file
Signatures	`personal_sign`	Proves ownership of a linked wallet
Signatures	ECDSA with EIP-712	Proves which university approved a batch
Commitments and proofs	Salted Keccak-256 commitments	Hide field values and stop guessing attacks
Commitments and proofs	Merkle trees	Reduce a whole batch to one on-chain root
Commitments and proofs	Selective disclosure	Proves chosen fields while hiding the rest

Initial Results

Results reported in the base paper

- A working prototype using a Solidity contract, IPFS and MetaMask on a local network
- Screenshots of issuing and verifying certificates
- No measurements of gas cost, latency, throughput or scalability

Baseline we measured on the paper's contract (local Ganache chain)

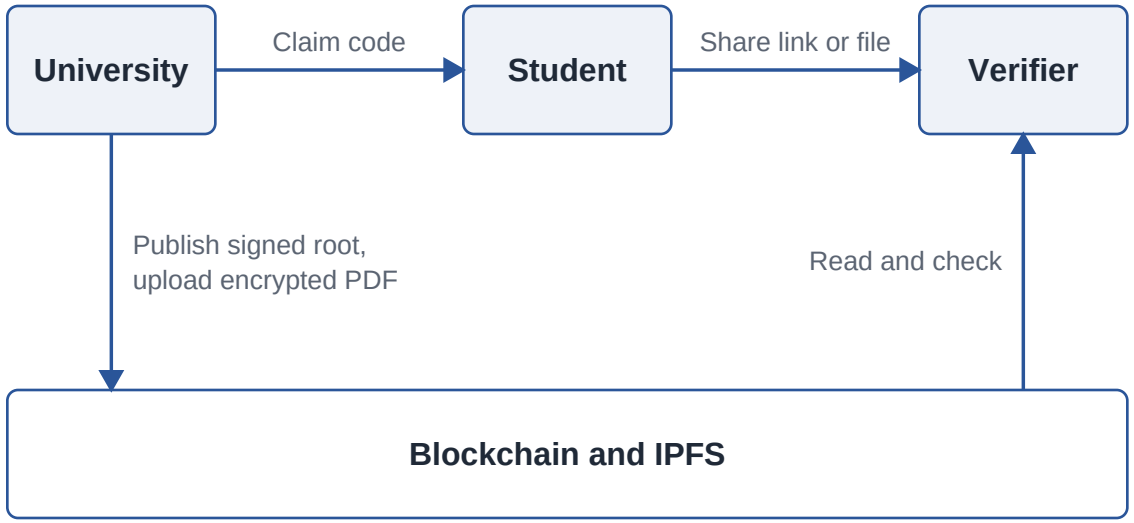
Action	Result
Deploy contract	1,421,842 gas
Issue 1 certificate	237,508 gas
Issue 500 certificates	about 118,754,000 gas (calculated)
Revoke a certificate	Not supported
Verify a certificate	Free, as no transaction is needed

Weakness of the Paper

Weakness	What it means
No issuer control	Any wallet can create a certificate under any university's name.
Personal data exposed	Names, UIDs and CGPAs are stored on a public chain in plain text and can't be erased.
Certificates can be listed	Contract events expose certificate IDs, and a public getter returns each record.
Unprotected files	PDFs are unencrypted on IPFS.
Weak verification	Finding a record proves only that it exists, not that a given file is genuine.
No revocation	Certificates can't be cancelled or corrected.
All-or-nothing disclosure	A student can't prove a degree without revealing everything else.
High issuing cost	Each certificate needs its own storage-heavy transaction.
No evaluation	The paper's claims aren't backed by measurements.

Improved Things

Weakness	Improvement
Anyone can issue	Registry of approved issuers and EIP-712 signatures
Personal data on-chain	Only a Merkle root goes on-chain
Guessable values	Random 32-byte salt for each field
Unencrypted PDFs	AES-256-GCM encryption before upload
Weak verification	Full check of proofs, issuer, chain record, revocation and PDF hash
No revocation	Revocation of one certificate or a whole batch
All-or-nothing disclosure	Field-by-field disclosure with Merkle proofs
One transaction per certificate	One transaction per batch
No evaluation	Measured gas costs and automated tests



We also built role-based dashboards with SQLite, CSV import and export, cost estimates, claim codes and share links.

Results

Action	Original (gas)	Improved (gas)
Issue 1 certificate	237,508	about 57,074 (about 76% less)
Issue 500 certificates	about 118,754,000	about 285,370 in 5 batches (about 99.8% less)
Revoke 1 certificate	Not supported	78,142–90,116
Revoke a whole batch	Not supported	75,499
Deploy contract	1,421,842	2,793,307
Verify a certificate	Free	Free

The extra deployment cost is recovered after about 6–8 certificates. The comparison isn't like for like, because the improved contract stores less on-chain and adds signatures and revocation.

Testing

28 contract and verification tests and 36 API tests pass.

Forged proofs, altered fields and PDFs, replayed or malleated signatures, unapproved issuers and access across roles are all rejected.

The tests also confirm that names, UIDs, courses and CIDs don't appear in the publishing transaction.

Conclusion

The base paper shows that Ethereum and IPFS can support certificate verification, but it leaves issuer control, privacy and revocation unaddressed. This project adds them using signatures, salted commitments, Merkle proofs and encryption, and it substantially reduces issuing costs.

Limitations

- The server can read the credentials it stores.
- The contract owner is a single point of trust.
- Salted hashes stay on-chain permanently.
- The system has only run locally, and the full flow hasn't been tested with real MetaMask.

Future work

- Zero-knowledge proofs
- Client-side credential encryption
- Multisig ownership
- Testnet evaluation